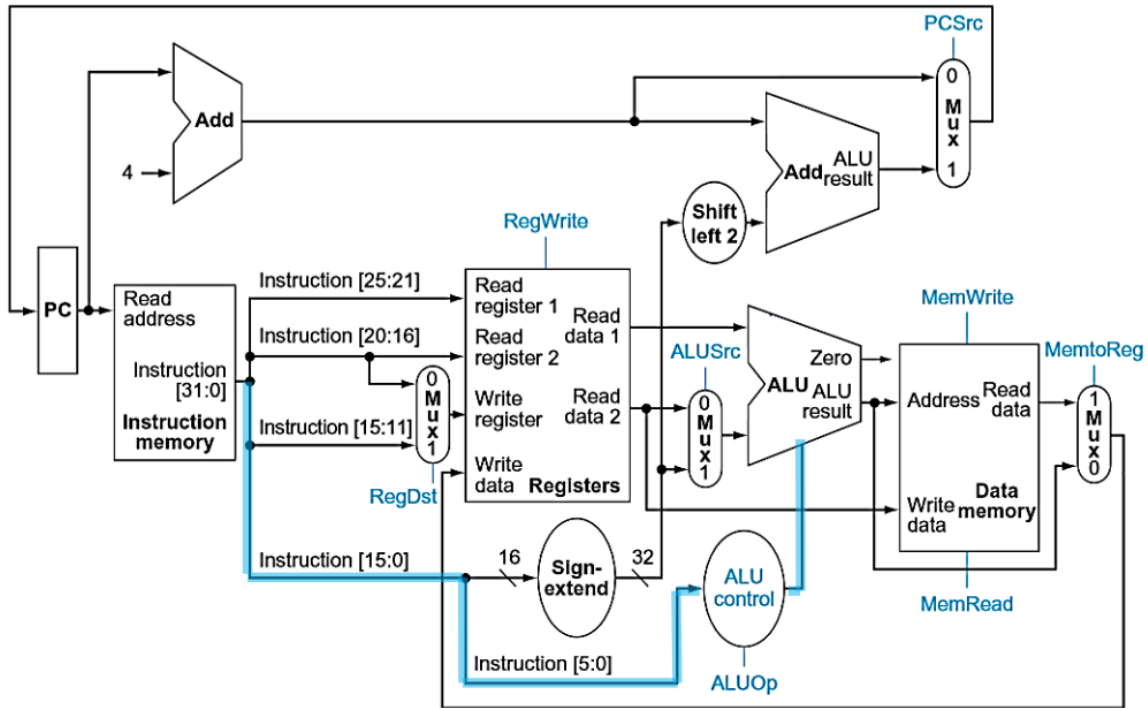


CE118-Lab06

Thiết kế vi xử lý (MIPS) đơn giản (tiếp theo)

1. Lý thuyết

Sinh viên ôn tập kiến thức về tập lệnh và datapath của MIPS.



Hình 1. Datapath tham khảo cho MIPS

Giới thiệu một số thành phần trong Datapath MIPS như:

- PC: Thanh ghi lưu địa chỉ lệnh – Program Counter.
- Instruction memory: Bộ nhớ lưu trữ lệnh (gọi tắt là IMEM).
- Sign extend: Mạch mở rộng dấu.
- ALU: Khối tính toán số học và logic. Ngoài ra còn có tín hiệu output (Zero) để cho biết kết quả của phép toán có bằng 0.
- Data memory: Bộ nhớ data, phục vụ lưu trữ trong quá trình xử lý lệnh (gọi tắt là DMEM).

- Các thành phần khác như Register, ALU, mạch cộng, ... đã được giới thiệu ở các bài Lab trước.

2. Thực hành

Sinh viên thực hiện thiết kế một vi xử lý đơn giản dùng để thực thi một số lệnh cơ bản trong kiến trúc MIPS (Thiết kế theo mô hình Controller & Datapath). Bộ vi xử lý này cần xử lý được một số lệnh MIPS cơ bản như sau:

STT	Lệnh	Loại lệnh	OPCODE / FUNCT
1	add	R	0 / 20 _{hex}
2	sub	R	0 / 22 _{hex}
3	and	R	0 / 24 _{hex}
4	or	R	0 / 25 _{hex}
5	nor	R	0 / 27 _{hex}
6	sll	R	0 / 00 _{hex}
7	srl	R	0 / 02 _{hex}
8	addi	I	8 _{hex}
9	ori	I	d _{hex}
10	andi	I	C _{hex}
11	lw	I	23 _{hex}
12	sw	I	2b _{hex}
13	bne	I	5 _{hex}
14	beq	I	4 _{hex}

Chi tiết về định dạng lệnh đọc thêm tại MIPS Reference Data.

Dưới đây là File assembly MIPS dùng để kiểm tra thiết kế và một số hướng dẫn thiết kế ban đầu.

```
addi $1, $0, 1
addi $2, $0, 2
addi $3, $0, 3
sll $4, $2, 2
add $5, $2, $3
ori $6, $4, 2
nor $7, $2, $5
addi $6, $5, -3
or $0, $0, $7
sub $0, $7, $1
srl $7, $7, 3
sw $4, 4($0)
lw $3, 4($0)
and $1, $0, $7
bne $7, $3, next
addi $1, $0, 7
addi $2, $0, 8
addi $3, $0, 9
next:
addi $4, $0, 10
beq $4, $5, next2
addi $5, $0, 11
addi $6, $0, 12
next2:
addi $7, $0, 13
addi $7, $0, 14
```

Sinh viên tiến hành **tạo (dump) file binary** từ đoạn code nói trên trong phần mềm MARS 4.5.

HƯỚNG DẪN THỰC HÀNH:

Lưu ý: Phần hướng dẫn chỉ mang tính chất tham khảo cho Datapath như hình 1, sinh viên có thể **tùy ý thay đổi/tối ưu Datapath** để phù hợp yêu cầu trong bài Lab này.

a. IMEM

Sinh viên tạo file verilog (.v) trên phần mềm với nội dung dưới đây để tạo ra khối IMEM.

```
module IMEM (
    input [4:0] addr,
    output reg [31:0] q
);

    reg [31:0] rom[31:0];

    initial begin
        $readmemb("binary.txt", rom);
    end

    always @(addr) begin
        q = rom[addr];
    end

endmodule
```

File “binary.txt” là kết quả của việc dump đoạn code đã cho trên phần mềm.

Ngõ ra của IMEM là 32 bit của câu lệnh. Chúng ta sẽ lựa chọn các trường bit cần thiết để sử dụng cho khối Sign Extend và Register.

b. Register: Đã hướng dẫn trong bài Lab trước.

c. Sign Extend: Để đơn giản, chúng ta cần mở rộng dấu của INST

c. DMEM: Thiết kế tương tự như Register. Lưu ý là DMEM có 2 tín hiệu riêng biệt là MemRead và MemWrite để điều khiển việc đọc/ghi (Register chỉ có 1 tín hiệu là RegWrite dùng để điều khiển ghi dữ liệu).

d. Datapath: Thực hiện ghép nối các khối cần thiết có được Datapath. Chúng ta có thể tái sử dụng các thiết kế 16 bit (Register, Mux, ALU...) từ các Lab trước đó bởi vì 16 bit là đủ sử dụng cho các phép tính toán trong đoạn code kiểm tra. Register phải có ít nhất là 7 thanh ghi chứa data (theo như đoạn code là \$1-\$7) và 1 thanh ghi Zero (\$0 – luôn luôn có giá trị là 0).

e. Controller: Từ các nhóm bit trong instruction, tiến hành thiết kế mạch điều khiển việc hoạt động các khối trong Datapath thông qua các tín hiệu như PCSrc, ALUSrc, ALU opcode, RegWrite...

f. Thiết kế bộ vi xử lý cho bài toán: Thực hiện ghép nối Controller và Datapath với nhau, sau đó mô phỏng kiểm tra hoạt động. Sinh viên quan sát và kiểm tra kết quả waveform (các phép toán, lưu trữ kết quả...)

Lưu ý: Phần hướng dẫn chỉ mang tính chất tham khảo cho Datapath như hình 1, sinh viên có thể **tùy ý thay đổi/tối ưu Datapath** để phù hợp yêu cầu trong bài Lab này.

Các nội dung cần chuẩn bị:

- File excel/word mô tả thiết kế luận lý của khối Controller.
- File/Sơ đồ/Hình vẽ mô tả khối Datapath mà mình sử dụng (trong trường hợp có thay đổi so với Datapath mẫu ở trên).
- Thiết kế và kết quả mô phỏng trên phần mềm (waveform cần phải thể hiện Clock, giá trị PC, kết quả từ khối ALU, các thanh ghi (\$0-\$7), .